

# C++ Notes

## Items 7 & 8 – Initialisation & nullptr

Adam HAMOUC

May 2026

### 1. Initialisation : (), =, {}

Trois syntaxes possibles en C++ moderne. = ne signifie **pas toujours** une assignation :

- Si l'objet **n'existe pas encore** → c'est une **initialisation** (appelle le copy ctor, pas operator=).
- Si l'objet **existe déjà** → c'est une **assignation** (appelle operator=).

```
Widget w1;           // default ctor
Widget w2 = w1;      // copy ctor (PAS operator=, w2 n'existait pas)
w1 = w2;             // operator= (w1 existait déjà)
```

### 2. Braced Initialization {} — Initialisation Uniforme

C++11 introduit {} comme syntaxe universelle. Comparaison des opérateurs selon le contexte :

- Membres non-statiques : {} et = OK, () interdit.
- Types non-copiables (std::atomic) : {} et () OK, = interdit.
- {} fonctionne **partout** : c'est son principal avantage.

**Avantages supplémentaires :**

- Interdit les conversions implicites narrowing → erreur de compilation.
- Immunité au *Most Vexing Parse* : Widget w() est une déclaration de fonction, Widget w{} est toujours un objet.

```
int x{ 3.7 };        // erreur : narrowing double -> int interdit
int y(3.7);          // OK (troncature silencieuse)

Widget w();           // PIEGE : declare une fonction, pas un objet !
Widget w{};           // OK : constructeur par défaut, toujours un objet
```

### 3. Pièges de {} : std::initializer\_list

Dès qu'un constructeur prend un std::initializer\_list, {} le préfère **toujours**, même si un autre constructeur est un meilleur match. Il tentera des conversions implicites pour y arriver.

Cas critique : auto + {} déduit std::initializer\_list, pas le type attendu.

```
// std::vector : cas le plus frequent en pratique
std::vector<int> v1(10, 20); // 10 elements valant 20
std::vector<int> v2{10, 20}; // 2 elements : 10 et 20

// auto + braces : piege
auto x = { 42 }; // std::initializer_list<int>, PAS un int !
auto y( 42 ); // int

// make_unique / make_shared utilisent () en interne et le documentent
```

## 4. Convention

Choisir une convention et s'y tenir. En **engine dev** : privilégier `()` par défaut pour éviter le piège `initializer_list`. Utiliser `{}` uniquement là où `()` est interdit ou pour initialiser des containers avec des valeurs spécifiques.

**Règle pratique** : être vigilant avec les classes qui ont un ctor `std::initializer_list` (ex. `std::vector`), pas avec `{}` en général.

## 5. nullptr vs 0 et NULL

0 est un `int`, pas un pointeur. `NULL` est un entier (souvent 0L). Le compilateur peut les interpréter comme pointeurs dans certains contextes, mais ce n'est qu'un fallback.

**Conséquence** : en cas d'overload `f(int)` / `f(void*)`, passer 0 ou `NULL` appelle `f(int)`, jamais `f(void*)`.

`nullptr` est de type `std::nullptr_t`, convertible implicitement vers **tous les types pointeurs**, mais jamais vers un type entier → appelle toujours le bon overload.

```
void f(int);      void f(void*);

f(0);            // appelle f(int)    -- PIEGE
f(NULL);         // appelle f(int)    -- PIEGE
f(nullptr);      // appelle f(void*) -- correct

// Avec auto : nullptr documente l'intention
auto result = findRecord(/* ... */);
if (result == nullptr) { /* clairement un pointeur */ }
if (result == 0)       { /* ambigu : int ou ptr ? */ }

// Avec templates : 0 et NULL deduits comme int -> erreur de type
template<typename F, typename P>
void call(F func, P ptr) { func(ptr); }

call(f, 0);      // ptr deduit comme int -> erreur
call(f, nullptr); // ptr deduit comme nullptr_t -> OK
```

## 6. Quand utiliser / éviter

`{}` : initialiser containers avec valeurs, membres non-statiques, éviter Most Vexing Parse.  
`()` : par défaut en engine dev, dans les templates, dès qu'une classe a un ctor `initializer_list`.  
`nullptr` : toujours, à la place de 0 et `NULL` — obligatoire dans les templates et avec `auto`.